Hochschule Karlsruhe
University of
Applied Sciences

Hochschule Karlsruhe

Fakultät Informatik und Wirtschaftsinformatik

# Protokoll – Game 1:
# Flip

Artem Kabin | 91506 | kaar1019

Rodion Strokatov | 91097 | stro1023

# Inhaltsverzeichnis

Kickoff-meeting (11.10.2026): ....2
Erste Entwicklungsphase ....5
Zweite Implementierungsphase ....6
Asset-Tabelle ....8
Nutzung von KI....9
Deployment am 29.10.2026 ....9

# Kickoff-meeting (11.10.2026):

Durch Erfahrung aus der Entwicklung des letzten Games, wurde sich darauf geeinigt dieselbe Engine und Deployment Methode zu verwenden wie in dem Game „Deceptive Wiring“. Das neue Game wird also erneut mit der neuesten Version der Godot Engine erstellt und über itch.io deployed.

**Gründe**: Das Team hat bereits kurze Erfahrungen mit der Godot Engine. Itch.io ist dabei sehr einfach einzurichten und bietet Tests auf dem produktiven Build. Obendrauf bietet Godot built-in MCP Server Endpunkte für eine Entwicklung mit LLMs, was die Entwicklungszeit beschleunigt und mehr Raum für Design und Entwicklungsentscheidung, sowie Testing bietet.

*Besprechung des Game Konzeptes:*

Die Gruppe entwickelte anhand einfacher, handgezeichneter Skizzen den Game Loop und den Namen der ersten Version des Spiels. Das Spiel soll dieses Mal einen Endless Loop beinhalten, damit das Spiel durch Design-Ideen „Wiederspielbarkeit“ erhält. Diese Entscheidung wurde kurz nach den Game-Tests des ersten Termins entschieden und richtet sich an Games, die tatsächlich nach den Tests wieder gespielt wurden. (REFERENZ SETZEN AUF DAS GEDACHTE GAME)

Das Spielprinzip wurde hiermit für beide Mitglieder verständlich erklärt. Es soll sich hierbei um einen Side-scrolling Plattformer, mit limitierten Controls und einfacher Kameraführung handeln, in dem Chunks (hier Level) nach und nach endlos und zufällig eingefügt werden:

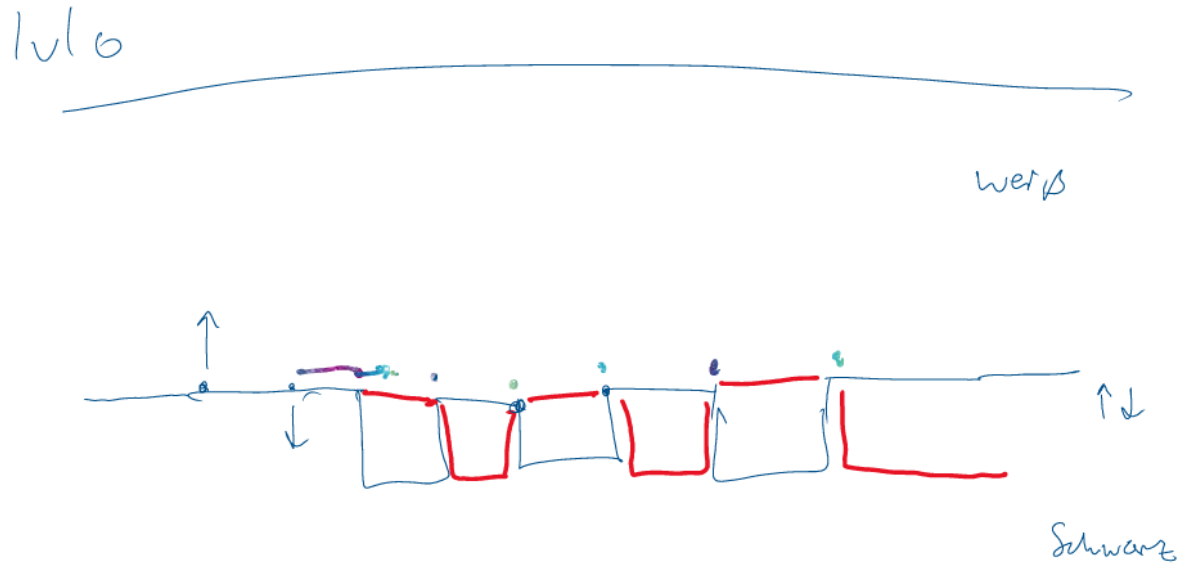



*Abbildung 1: Einfacher Chunk (Kickoff-Meeting 1)*

Der Spieler befindet sich auf einer Linie, die die Mitte des IFrames horizontal halbiert. Der Spieler läuft konstant auf der Linie entlang nach rechts, wobei Dieser nicht springen kann. Die Pfeile zeigen, dass der Spieler nur die Möglichkeit besitzt nach rechts zu laufen oder „die Seite zu wechseln“. Das Spiel bekam durch dieses Prinzip den schlichten Namen **FLIP.** Diese Funktion erlaubt dem Nutzer zu jeder Zeit die Spielumgebung durch Knopfdruck anzupassen. Da der Spieler nicht in der Lage über Hindernisse wie Löcher auf der Linie mit Sprüngen zu umgehen, muss der Spieler stattdessen über den Knopfdruck die Seite wechseln, wodurch Löcher zu Plattformen und Plattformen zu Löchern werden. Diese Control muss auf der richtigen Position durchgeführt werden, um beim Wechsel nicht herunterzufallen und ein Game Over auszulösen.

Die Game Idee wurde von beiden Entwicklern verstanden und akzeptiert. Danach wurde sich darauf konzentriert mit diesem Konzept das Game Thema zu treffen.

*Anpassung auf das Game Thema:*

Das Thema dieses Spiels war es den Spieler aktiv zum Zögern zu bringen. Daher wird das Game um weitere Mechaniken erweitert. Der Spieler erhält die Möglichkeit, jederzeit anzuhalten, oder weiterzugehen. In der späteren Entwicklung wurde auch eine Beschleunigung hinzugefügt, da die Entwickler aus eigener Erfahrung behaupten, dass Geschwindigkeit ein besseres UX bieten als ständiges Warten oder Zögern.

Dazu wurde das Thema und dessen Beschreibung bewusst anders interpretiert, als vermutlich die anderen Entwickler verwenden werden. Wir konzentrierten uns hier nicht auf Zögern, indem man den Spieler in einen Angst Zustand versetzt. Zögern kann nämlich ebenfalls durch Unsicherheit verursacht werden. Deshalb ist der Spieler nie in der Lage die andere Flip-Seite der Karte zu sehen, ohne zuvor die Seite gewechselt zu haben. Das ermöglichte dem Team, von der offensichtlichen Horror-Game Lösung auszuweichen und weitere Features für das exakte „Zögern durch Unsicherheit“ zu überlegen:

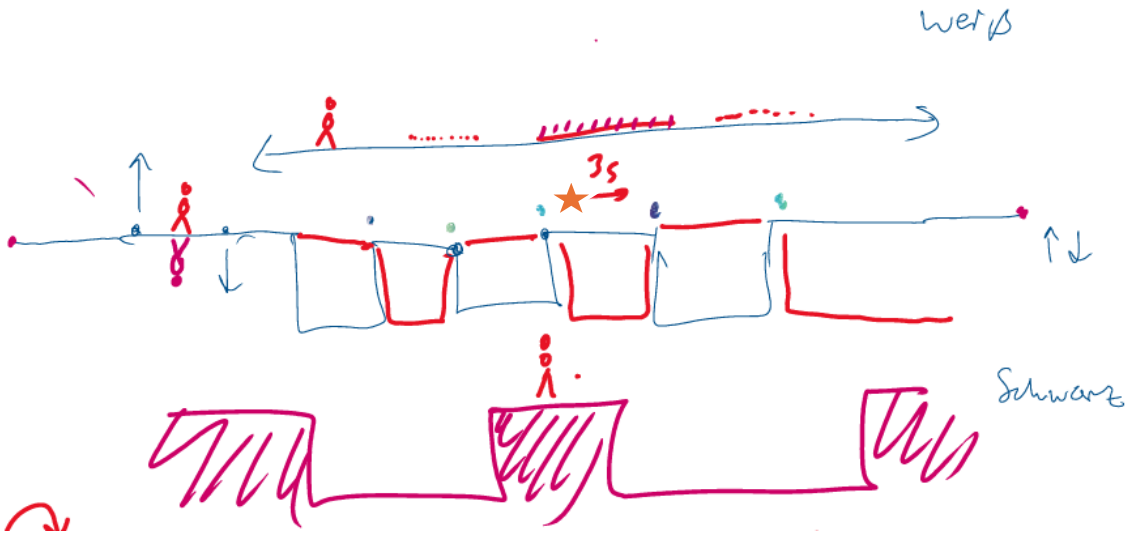



*Abbildung 2: Chunk mit Stern (Kickoff-Meeting 1)*

In der Zeichnung wurde zur Visualisierung ein Stern gezeichnet und bewegt, um das neue Features zu verdeutlichen. Es soll zu den Plattformen, die es nur auf einer der beiden Seiten gibt, auch Gegner auf Höhe des Spielers geben, die es ebenfalls nur auf einer Seite gibt, um den Tester in ein Game Over zu bringen. Diesen können wir außerdem eine Bewegung innerhalb des Chunks geben. Diese Möglichkeit erlaubt es uns, bei Positionen, in denen der Spieler nicht einfach wechseln kann, da unter ihm dann die Plattform verschwindet, ebenfalls sich zu merken, wo dieser Stern sich befindet, bevor er aktiv wechselt. Später in den ersten Tests, wird ein Anhalten, lauschen, und ein Rythmusgefühl für die Bewegung des „Sterns“ bekommen zu einer wichtigen Aufgabe in diesem Spiel gewertet.

Diese Sterne wurden erweitert durch eine letzte Möglichkeit, ein Game Over zu verursachen. Die sogenannten „Hunter“, die aus einer Dr Who Folge „Weeping Angels“ abgekupfert wurden sind in der Lage den Spieler zu verfolgen, aber nur solange man diese nicht sieht. Das bedeutet, dass diese Form von Sternen auf der einen Seite stehen bleiben und auf der anderen, unsichtbaren Seite bereits in dem Spieler drin sein könnten. Das bedeutet, dass der Spieler nicht direkt an der nächsten Kante schon stehen sollte, wo dieser durch Plattformwechsel gezwungen ist zu flippen, sondern etwas weiter zuvor, damit er Distanz zu diesem Hunter gewinnt. Natürlich bedeutet das ein Deadlock, sollte das nicht gemacht worden sein. Das ist bewusst so implementiert worden. Hier die Zeichnung zu der Idee (Bewegung auf den Spieler zu wurde im Meeting mit der Steuerung der Maus durchgeführt):




*Abbildung 3: Hunter Chunks (Kickoff-Meeting 1)*

Das Meeting wurde mit der Planung dieser Features erfolgreich beendet.

## Erste Entwicklungsphase

### *16.10. Erster Prototyp*

Zunächst wurde eine Kopie des ersten Spieles erstellt. Alle Level-Szenen, der Spieler, die Menüs, das Game Over und die State-Machine wurden als Basis komplett übernommen. Danach wurden diese angepasst und erweitert. Darunter befanden sich Controls mit A und D und das Verwerfen von vertikalen Bewegungen. Fall Physics mit Gewicht wurden hinzugefügt, sind aber in der Entwicklung unbrauchbar, da bei fehlendem Boden einfach auf ein Shape mit Collidor gesetzt wurde, die ein Game Over auslöst. Der F Button wurde für den Welt-Wechsel eingeführt. Obendrauf wurden komplexere Mechaniken aus einem YouTube Tutorial, das die beiden Entwickler schon zur Entwicklung des letzten Games gesehen haben, eingeführt: Eine Coyote-Time, also ein Zeitfenster von (bei uns) 0,12s, in der der Spieler beim Wechsel auf eine andere Kante noch Zeit hat, bevor dieser davon stirbt. Ein fester Cooldown Wert für den Flip mit F wurde ebenfalls eingefügt.

Zuallererst wurde ein Level System gebaut, ohne Chunk Loading zu verwenden und alle Mechaniken zu testen. Daher wurde eine einfache Szene mit einem statischen Stern (Kugel) gebaut und ein paar Abgründen.

Danach wurden alle Reste aus „Deceptive Wiring“ (Spiel 1) entfernt und das Testing Level auch. Danach entstand das Konzept des Endless Runners, wobei der Spieler nicht mehr kontrolliert, sondern automatisch nach rechts läuft. Dazu wurden dann weitere Chunks erstellt mit weniger und mehr Lücken und mit Variation der Größe der Plattformen. Um die Schwierigkeit des Spiels weiter zu erhöhen, haben wir außerdem eine Beschleunigung eingebaut. Der Spieler beschleunigt im Laufe des Spiels von 140px/s hoch auf maximal 380px/s. Ein Score System, das die durchlaufenen Pixel zählt, ist ebenfalls drin (Score = 100px/100 = 1). Hinzu wurde eine Wand von links implementiert, die den Spieler verfolgt, diese Mechanik wird im nächsten Meeting besprochen und letztendlich verworfen.

Weitere Kleinigkeiten wurden eingefügt:

- Sterne in Chunk-Lücken (später entfernt, durch Implementierung von Save Zones)
- Todesursache im Game Over Screen
- Background Skript und HUD für Erkennbaren Flip (Dunkel Hell Kontrast-Wechsel)

Die Entwickler konnten ab dieser Etappe sehr schnell neue Features und chunks im 2D Scene Creator erstellen durch die erfolgreiche Implementierung und Besprechung des Workflows vom chunkloader.

### *19.10 Meeting 2*

Ein weiteres Meeting zur Besprechung des derzeitigen Standes wurde angesetzt. In Diesem wurden außerdem die nicht besprochenen Entscheidungen noch einmal überdacht. Die Wand zum Beispiel setzt den Spieler unter Druck, was ihm nicht genug Zeit bietet, um in einen Warte Zustand, also zum Zögern zu kommen, weshalb diese wegfallen sollte. Dazu gibt es bereits eine Funktion, die den Druck und damit auch die Schwierigkeit erhöht, unzwar die Geschwindigkeit des Players. Weitere Bugs, die durch die Wand entstanden sind, sollen danach ebenfalls aufgeräumt werden. Die alten Lücken-Chunks sollen noch einmal überarbeitet werden, da hier ohne Sprungmöglichkeiten diese Hindernisse nicht passierbar seien. Daher wurden auch hier die Löcher mit echten Löchern und umschaltbaren Collidorn angewandt. Man konnte schlichtweg die visibility und die Kollision der Löcher und der sichtbaren Plattformen per Code umschalten.

In dem Meeting wurde obendrauf der Scope der gesamten Implementierungsphase besprochen. Zu dem abgeschlossenen Game mit einigen Chunks und den Stern Features sollen auch Assets und Sounds benutzt werden. Die Gruppe setzte dieses Mal mehr Wert auf kleinere Dinge, um die User Experience zu maximieren. Nach Besprechung der Sachlage und Aufrärumen der Unklarheiten ging es dann schon in die zweite Phase.

## Zweite Implementierungsphase

Die eingeführten Logiken wurden zunächst entfernt, die so im letzteren Meeting nicht besprochen wurden oder das Spielprinzip oder das Thema nicht treffen. Lücken Chunks Problem wurde ebenfalls behoben durch virtuelle Löcher. Damit kann man Löcher zeichnen und mit Area2D-Kill-Zone als Parameter belegen. Daher ist die Chunk Entwicklung immer nur eine Erstellung der Szene mit Szenencreator Tool von Godot. Dazu wurde zur Coyote Time (durch Versetzung der Zone um ein paar Pixel nach unten) eine 14px Gnade Zone erstellt, da diese Breite groß genug ist, damit der Spieler noch steht und ein weiteres Zeitfenster zum Flippen bietet. Der Flip Cooldown wurde ebenfalls reduziert.

Um später mit den Hunter Sternen eine einfachere Implementierung zu bieten, wurden Save Zones eingeführt, die mit Licht markiert werden. Diese verändern sich bei Flips nie. Das Problem hierbei ist, dass es nicht offensichtlich ist, dass diese Zonen sicher sind, also wurde eine Hilfemöglichkeit in das Backlog eingetragen und in der Letzten Phase implementiert.

Ab diesem Zeitpunkt wurde erneut auf die Dokumentation und dieses Mal auch auf die Hilfe von Claude Code OPUS 5.5 zur Implementierung der weiteren Features gesetzt. Die Prompts wurden meist übernommen, von den Entwicklern geprüft und getestet.

Es wurde ein Prompt geschrieben, um „Kugeln“ oder später Hazard Chunks zu implementieren:

Prompt 1:

„Die Kugeln sollen auf Höhe des Spielers sein und auch nur in einer Farbe existieren (egal welche) Diese sollen den Gegner auf der sichtbaren Farbe umbringen, sollte der Spieler mit den Kugeln kollidieren. Auf der "unsichtbaren" Flip-Seite machen diese nichts und sind auch nicht sichtbar. Es soll noch dazu Chunks geben, in denen sich die Kugeln von links nach rechts und zurück in einer gewählten Strecke mit einer konstanten geschwindigkeit bewegen.“

Hier wurde implizit auf die Verwendung der Sterne aus der letzten Implementierung gesetzt. Claude hat hierbei schlichtweg die Implementierung der Sterne aus benutztem Code wiederverwendet und diesen Strecken und eine Geschwindigkeit gegeben. Im Commit kam folgendes von Claude heraus:

“Spheres exist in one world only (in_dark_world), sit at player height on a floor solid in both worlds, and are visible and deadly only there. They may patrol a chosen distance at constant speed. Sphere chunks (sphere_chunk.gd) are entries in the hazard pool in game.gd next to the hole patterns.”

Dazu hat Claude ebenfalls einen Bug entdeckt und diesen direkt aufgeräumt:

„Restart now waits one physics frame between teleporting the player and spawning new chunks; otherwise a new kill zone could overlap the old death position and kill again.”

Prompt 2:

“Der Spieler wird hierbei noch nicht zögern, da er sich konstant nach vorne bewegt und nur zum richtigen Zeitpunkt flippen muss. Wir implementieren nun eine Mechanic, die den Spieler weiter zum Zögern bringt. Implementiere eine Sichtweite, die initial im Maximum liegt und immer kleiner wird je weiter sich der Spieler bewegt. Die Reichweite wird wieder größer, wenn der Spieler anhält. Wir haben bereits Rauten verwendet, um safe-Plattformen zu markieren. Kannst du diese Rauten noch dauerhaft zum Leuchten bringen, sodass wenigstens immer zu sehen ist, wo der Spieler kurz und sicher anhalten darf?“

Dieses Feature soll den Spieler weiter erzwingen, zu zögern, da blindheit bei den meisten Menschen erneut eine Unsicherheit und damit ein Zöger-Verhalten auslöst:

„feat(flip): sight radius that shrinks with distance travelled, glowing safe markers above the fog

A radial fog (sight_fog.gd, sight_fog.gdshader) follows the player as a child node and hides everything beyond the sight radius under the current background colour. The radius starts at 340 px, loses 0.35 px per px travelled (max to min after about 715 px) and regrows at 220 px/s while the player stands still, down to a minimum of 90 px. The fog colour tweens with the world flip like the background.

The radius logic runs in _physics_process: the player only moves on physics ticks, so measuring in _process would count most frames as standing and regrow the radius while running.

Safe-zone markers glow permanently with a faint disc and two breathing rings and render above the fog (z_index 60 vs 50), so the player always sees where stopping is safe. The drawn diamond stays a placeholder; the glow survives an asset swap.

Start hint updated. Decision log entry 0025 added.”

Die genannten Rauten waren die Markierungen über den Save Zones. Licht in Horror Games gibt es Spieler i.d.R. ein Gefühl von Sicherheit.

Dazu wurden immer mehr Chunks implementiert und commited. Danach wurde Recherche für die Erstellung von Assets, also Sprites und Sounds durchgeführt. Die Entwickler verwendeten im Folgenden selbst erstellten Sprites mithilfe von der „Aseprite“ Software und Sounds aus Pixabay.com. Dabei wurde bei den Sounds immer darauf geachtet, dass diese frei und kommerziell benutzbar sind (hinterlegt durch ein Schild-Icon bei den Sounds.

Die Sprites wurden angelehnt auf ein Horror Theme, speziell „Scooby-Doo“ und erhält eine kurze Story im Laufe der späteren Implementierung und einem Setting von einem endlosen Gang in einem verlassenen Hotel. Dazu wurden die Menüs von Schriftbasiert auf Sprites und Assets umgestellt und ein Einstellungs- und ein Hilfe-Fenster implementiert, um das Spiel komplett abzurunden. Der Spieler ist dann in der Lage, Sounds, Fenstermodus und sogar die Controls anzupassen.  Ein Scoreboard, das im Browser des Clients die Bestenliste über eine Session behält ist ebenfalls eingeführt worden (Ausweichmöglichkeit durch Einschränkung von keinen Datenbanken)

Alle Assets liegen neben diesem Protokoll in einer entsprechenden ZIP-File:

## Asset-Tabelle

| Sound | Titel | Ersteller | Nutzen |
|---|---|---|---|
| Flip | Elemental magic spell impact outgoing | rescopicsound | Sound beim Flippen (overlapping |
| Ghost hunt | Ghost horror sound | Dragon studio | Lauschen, ob ein Hunter in der Nähe ist (intervall) |
| Fall | Creepy fall | universfield | Tod durch Fallen |
| Ghost hit | Footsteps walking | universfield | Tod durch Geist (zuvor Stern) |
| Steps | Horror impact hit | universfield | Rennen, Loop parallel zur Geschwindigkeit des Spielers |
| lantern | Fire sounds | Dragon studio | Loop, Feuergeräusche in der Nähe von Laternen (Sicherheit) |
| Ui back | Ui close sfx | litupsubway | UI zurück button und ESCAPE Knopf |
| Ui move | Click button | universfield | Navigation in UI |
| Ui select | Ui hover sfx | litupsubway | Selektion in UI |

| | | | |
|---|---|---|---|
| Game over | Biodynamic impact braam tonal dark | bryansantosbreton | Game Over Sound |
| Record | Magic surprise | universfield | Überlappender Ton für neuen Rekord, für helleren Ton |
| Run | Dead cinematic dark ambient | everything is dead | Musik-Loop ingame |
| menu | Dark ambient soundscape | everything is dead | Musik-Loop in Menüs |

Die Sounds wurden im Nachhinein zu OCG oder WAV Files umformatiert, für einen komprimierteren build passend zu Web Export.

## Nutzung von KI

Die Einzige verwendete KI ist Anthropics Claude Code Modell Opus 5.5 im Max 200 Plan (mittleres Thinking). Prompts und deren Output wurden hier ausführlich dokumentiert. Alle weiteren Implementierungen enstanden aus eigener Hand ohne direkte Nutzung von Tutorials oder Foren. Sounds sind das Einzige, was von Dritten benutzt worden ist.

## Deployment am 29.10.2026

Das Game wurde standardgemäß (wie im letzten Game) mit allen verwendeten Assets und einem index.html export gebaut und auf itch.io deployed und wird von den itch.io gehostet. Ein finaler Game Test wurde von beiden Entwicklern durchgeführt und das Game abgewunken